Ninja also provides a class that translates a Django ORM object into a `Schema` for serialization. You do this by subclassing a `ModelSchema` and configuring it based on your ORM Model class. This is similar to the `ModelForm` class discussed in chapter 7.

Consider, once again, a mobile application, this time with a screen that displays the promoters in your database. To do this, you'll need to add some API endpoints to the RiffMates `promoters` Django app. First off, create RiffMates/promoters/api.py. Inside this file, you declare `PromoterSchema`, which extends a Ninja `ModelSchema`. This class uses an inner class, called `Meta`, whose properties declare how the serialization works. At minimum, you need two properties: `model` points to the `Model` class to be serialized, and `fields` is a list of the fields to participate in the serialization. As its name implies, the `PromoterSchema` class gets used to serialize a `Promoter Model`. Note that the `Promoter` class got modified in the chapter 10 exercises; the example presented here is based on that version. If you didn't do the exercises, you'll need different values in the `fields` property.

> **NOTE** In early versions of Ninja, `Meta` was called `Config`, and `fields` was called `model_fields`. With the 1.0 release, the names were made consistent with Django's own ORM models

Once you've declared `PromoterSchema`, you use it with a `response` argument in a Ninja decorator. This informs Ninja that the schema gets used for serialization of the endpoint's return value.

The promoters screen in your mobile application would contain a list of promoters, similar to the list of songs mentioned before. The `promoters()` endpoint, shown in the following listing, returns a list of all the promoters used to populate that screen. You tell Ninja that a function endpoint returns a list by using Python type hints. With the correct type hint in place, all the function needs to do is return a `QuerySet` of `Promoter` objects, and Ninja takes care of the rest.

Listing 11.4 Serialize an ORM Model class with Ninja `ModelSchema`

```
# RiffMates/promoters/api.py
from ninja import Router, ModelSchema

from promoters.models import Promoter

router = Router()

class PromoterSchema(ModelSchema):
    class Meta:
        model = Promoter
        fields = ["id", "full_name", "birth", "death"]

@router.get("/promoters/",
        response=list[PromoterSchema])
def promoters(request):
    return Promoter.objects.all()
```

Each api.py file declares its own Router object.

Subclassing ModelSchema creates a serializer for your Django ORM Model.

This class serializes a Promoter object.

List the fields to participate in the serialization.

Tell Ninja that this endpoint returns a list of PromoterSchema objects.

Return all the Promoter objects.